RTOS 异构快启
开发指南

版本号: 1.0

发布日期: 2024.03.13

广州星翼电子科技有限公司zdyzlinux

版本历史

版本号	日期	制/修订人	内容描述
1.0	2024.03.13	KPA0576	建立初版

广州星翼电子科技有限公司zdyzlinux

目　录

1 前言 **1**
1.1 文档简介 1
1.2 目标读者 1
1.3 适用范围 1
1.4 相关术语介绍 1

2 模块介绍 **3**
2.1 模块功能介绍 3
2.1.1 快速倒车介绍 3
2.1.2 RISCV 快速动画介绍 4
2.1.2.1 分辨率自适应功能 5
2.1.2.2 支持替换动画文件 5
2.1.3 异构核固件加载引导流程介绍 8
2.1.4 UBOOT 显示图层同步介绍 10
2.1.4.1 DISP 驱动版本 10
2.1.4.2 DRM 驱动版本 12
2.1.5 RISCV 快启任务及线程介绍 13
2.1.6 buf 数据流介绍 15
2.1.6.1 快速倒车 15
2.1.6.2 快速动画 16
2.1.7 倒车轨迹线 16
2.2 源码结构介绍 17
2.3 模块配置介绍 18
2.3.1 rtos menuconfig 配置 18
2.3.2 linux 平台配置说明 20
2.3.3 模块驱动配置 23
2.4 公共接口 API 介绍 25
2.4.1 视频输入回调 25
2.4.2 内存相关接口 26
2.4.3 显示预览相关接口 27

3 FAQ **29**
3.1 倒车状态节点 29
3.2 新增快启模块 29
3.3 新增快启版型 29
3.4 替换快启 camera 30
3.5 快启时间统计 31
3.6 TWI 通信失败 32

版权所有 © 珠海全志科技股份有限公司。保留一切权利

插　图

图 2-1　快速倒车框图 4
图 2-2　快速动画流程 5
图 2-3　boot0_package 6
图 2-4　打包脚本 6
图 2-5　快启动画流程 7
图 2-6　rv 快启动流程 8
图 2-7　boot_package.cfg 配置 9
图 2-8　uboot 图层信息同步 11
图 2-9　drm 版本显示控制权管理 12
图 2-10　快启任务间消息处理 14
图 2-11　倒车 buf 执行流 15
图 2-12　快速动画 buf 执行流 16
图 3-1　boot0 引导 freertos 时间戳 32
图 3-2　rv 送显时间戳 32

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

1 前言

1.1 文档简介

介绍 SUNXI 平台 riscv 快速启动的流程架构，指导其他定制功能的快速添加。

1.2 目标读者

- 快速启动（包括快速倒车/快速动画）开发人员/维护人
- 快速启动的应用层使用者

1.3 适用范围

表 1-1: 适用产品列表

ic 平台	内核版本	驱动文件路径
T113	linux5.4 /linux5.15	rtos/lichee/rtos-hal/car_reverse
T527	linux5.15	longan/rtos/lichee/rtos-hal/car_reverse
T733/T736	linux5.15	longan/rtos/lichee/rtos-hal/car_reverse

1.4 相关术语介绍

表 1-2: 术语介绍

术语	解释
CSI	Camera Signal Input 指 soc 内部视频信号输入模块
TVD	Television Decoder 电视解码

版权所有 © 珠海全志科技股份有限公司。保留一切权利

术语	解释
CVBS	Composite Video Broadcast Signal 复合视频广播信号
NTSC	National Television Standards Committee , 一种分辨率
PAL	Phase Alternating Line , 一种分辨率
DI	Deinterlace，去隔行模块，用于增强图像的画质
G2D	图形加速单元，用于画面的旋转和缩放
DISP2	全志平台的显示框架
VIPP	视频输入内部硬件模块，主要用作缩放

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2 模块介绍

2.1 模块功能介绍

快速启动的设计初衷是在将一些需要快速运行的业务放在同异构小核中去提早执行，比如快速倒车中实现快速出图，快速动画中实现快速解码等等，但不限于倒车和动画，快启流程可以满足客户定制化的快启需求，本文档简单介绍快速倒车和快速动画基本流程，从这两个公版 sdk 的案例中介绍快速启动的架构设计思想，让客户能够快速把自己业务集成到该流程中。

文档章节划分如下：

- 模块功能介绍：该章节介绍快速倒车和快速动画的基本概念，以及一些关键流程；
- 源码结构介绍：介绍快启业务代码目录层级；
- 模块配置介绍：介绍快启流程业务的可选配置项；
- 公共接口 API 介绍：介绍快启动流程常用公共接口；
- FAQ: 介绍一些常见问题处理方法。
- **注意**：不同平台所使用的同异构核心架构是不同的，具体区分如下表：

ic 平台	快速启动核心	已适配功能
T113	riscv c906	快速动画/快速倒车
T527	riscv e907	快速倒车
A733/T736	cortext-a73	快速倒车

2.1.1 快速倒车介绍

快速倒车继承于旧平台的内核倒车，把整体流程移植到 riscv 或隔离 a 核的 freertos 系统中运行，主体的运行流程围绕着 csi，display，g2d，deinterlace 模块驱动运行，即通过 csi 采集到 yuv 数据后通过 g2d 旋转，deinterlace 去隔行后（pal/ntsc 隔行信号)，将 camera 数据送到 display engine 图层预览显示，相比内核倒车 rv 倒车有一些特殊流程处理，比如模块的控制权和 linux 内核的相互流转，与 uboot 的图层信息同步等，可以见以下章节介绍或《RISCV 快速倒车概要设计文档》。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

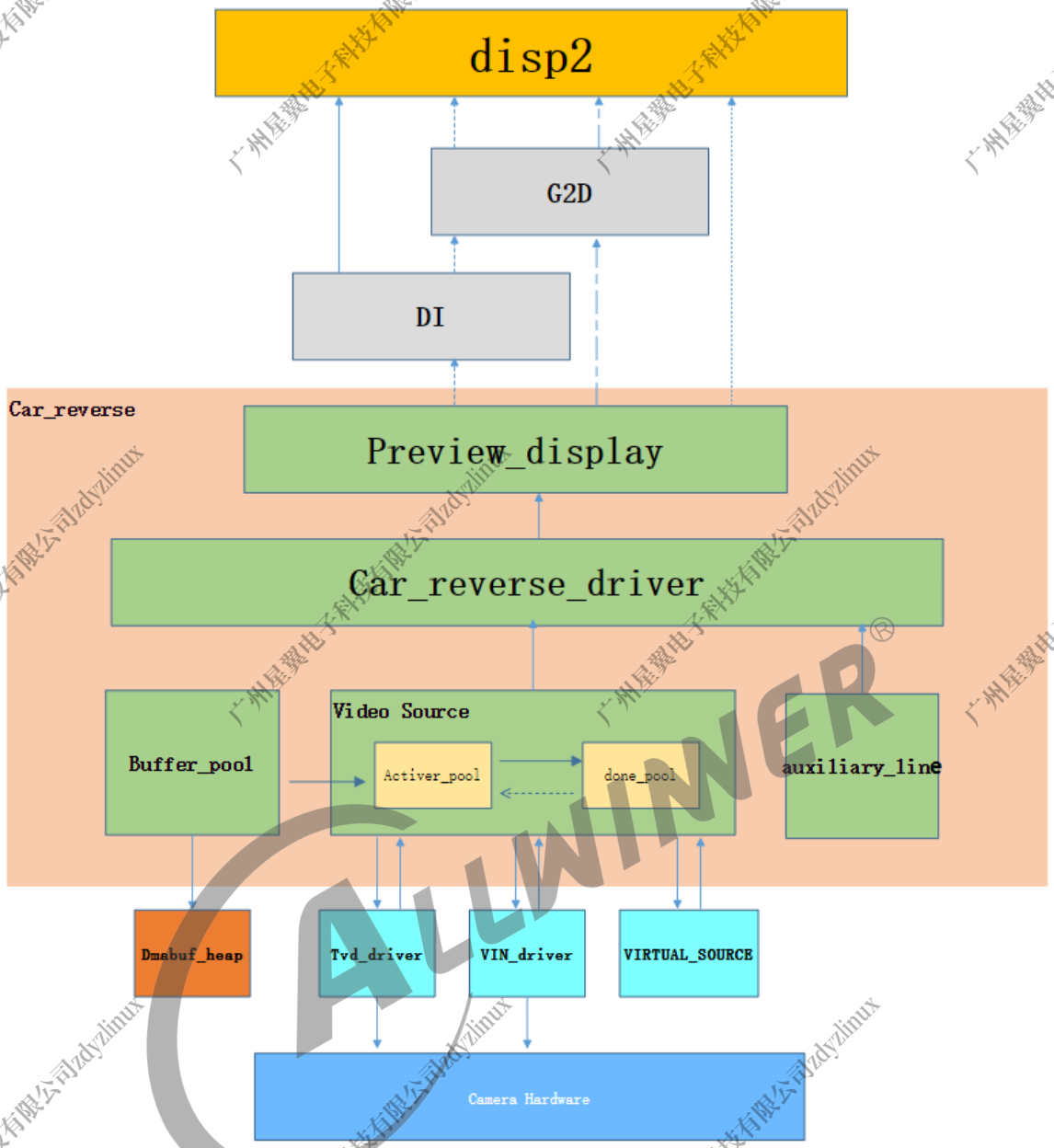

图 2-1: 快速倒车框图

2.1.2 RISCV 快速动画介绍

riscv 快速动画指的是上电时在 rv freertos 将 boot_package 的 h264 动画码流文件提取后解码成 yuv 后，显示视频动画于屏幕上的流程，软件框图如下：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

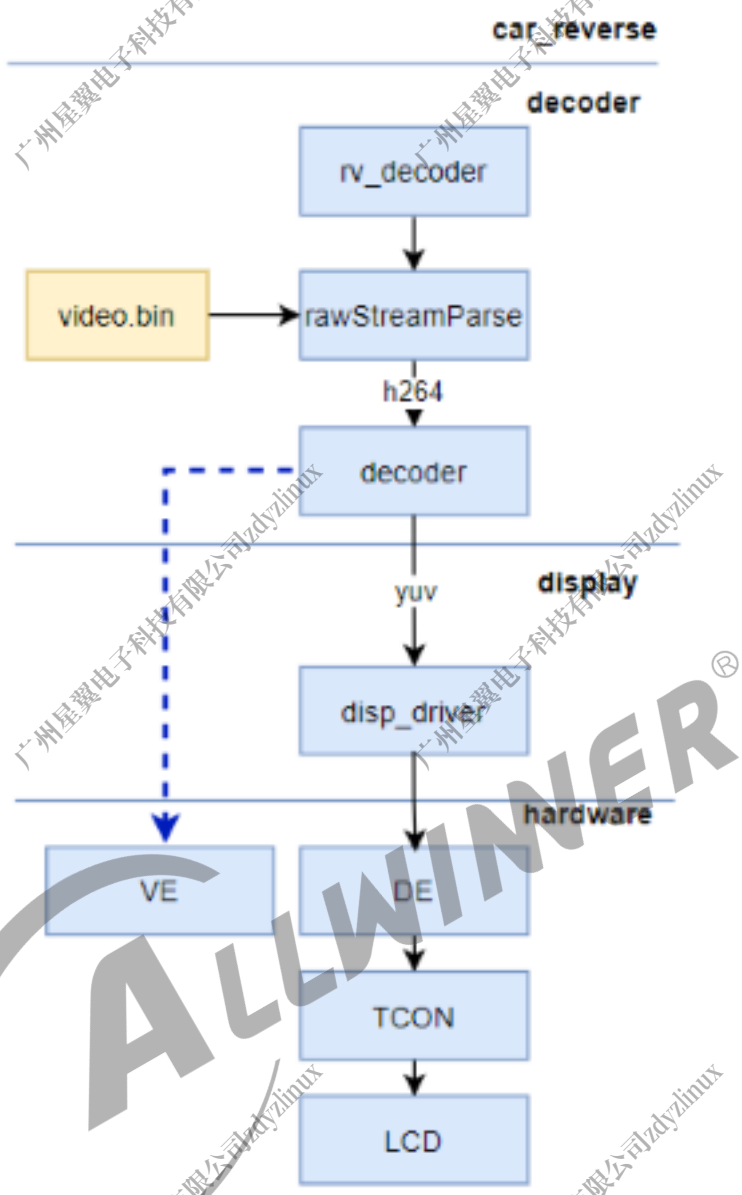

图 2-2: 快速动画流程

2.1.2.1 分辨率自适应功能

解码器支持识别码流文件的分辨率大小，无需软件修改解码器的输入输出参数。

2.1.2.2 支持替换动画文件

替换动画文件的步骤以及打包脚本, 以 s4 版型为例：

- 替换 sdk 下 device/config/chips/T113_s4(以 T113 为例)/configs/

版权所有 © 珠海全志科技股份有限公司。保留一切权利

evb1_auto_fastboot_video(特定版型)/bin 下的 video.bin，注意仅支持 h265 文件。

- 打包脚本配置修改 device/config/chips/T113_s4(以 T113 为例)/configs/evb1_auto_fastboot_video(特定版型) 下的 boot0_package.fex

```
[package]
;item=Item_TOC_name,          Item_filename,
item=u-boot,            u-boot.fex
item=optee,             optee.fex
item=melis-elf,         amp_rv0.fex
item=video,             video.fex
;item=scp,             scp.fex
;item=logo,                  bootlogo.bmp.lzma
;item=shutdowncharge,        bempty.bmp.lzma
;item=androidcharge,         battery_charge.bmp.lzma
item=dtb,                   sunxi.fex
```

图 2-3: boot0_package

- build/pack 脚本 boot_file_test 新增

```
${LICHEE_CHIP_CONFIG_DIR}/\${BIN_PATH}/video.bin:${LICHEE_PACK_OUT_DIR}/video.fex
```

图 2-4: 打包脚本

版权所有 © 珠海全志科技股份有限公司。保留一切权利

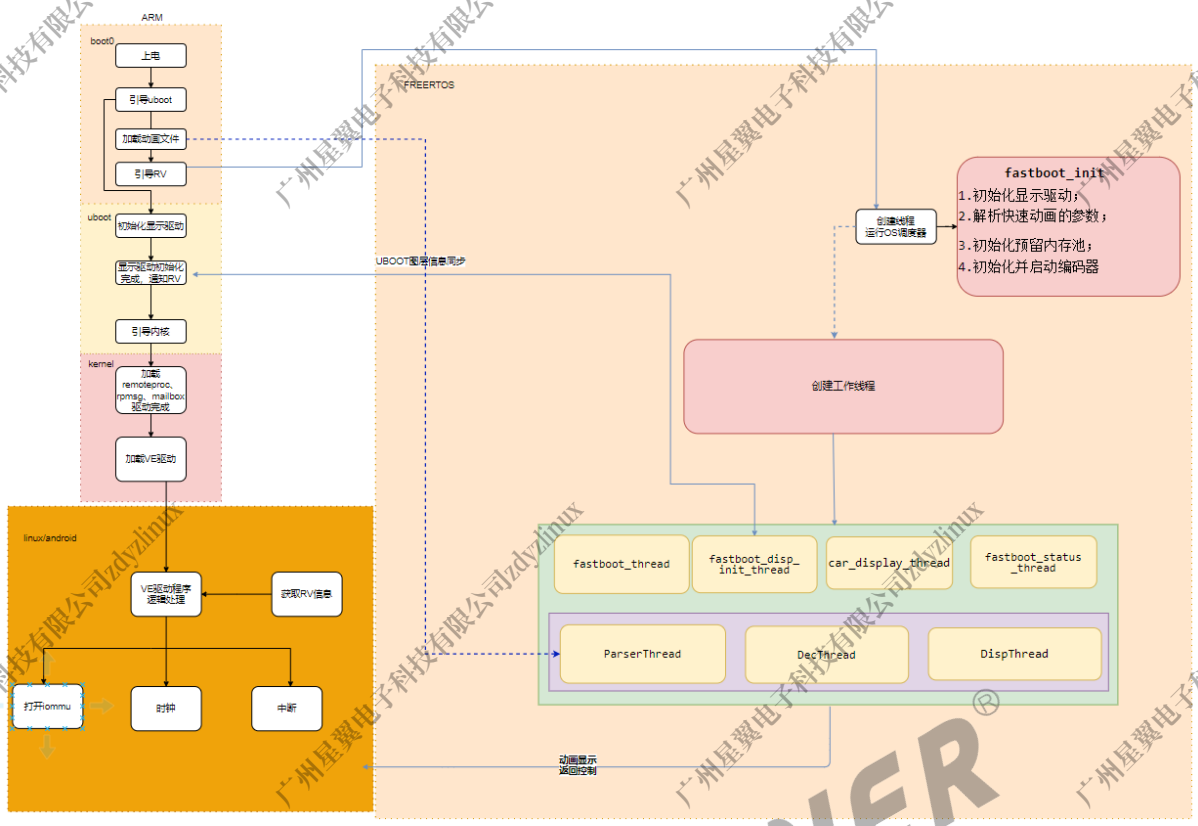

图 2-5: 快启动画流程

快速动画的流程如上图，boot0 阶段把 rv 固件引导，同时把 video.bin 的码流文件，加载到对应预留的 ddr 内存中，在 fastboot_init 接口中，初始化开机动画必要资源。

- 同步来自 uboot 的显示图层信息；
- 解析快速动画的配置；
- 初始化预留内存的内存池；
- 初始化解码器并启动

接着是启动快速倒车/动画各个工作线程：

- fastboot_thread: 主要是接受 CAR_VIDEO_START 的消息，这时候后会 preview_output_init 启动显示；
- fastboot_disp_init_thread: 主要是与 uboot 交换显示图层信息；
- car_display_thread：送显线程，目前动画送显还是再解码码模块内部操作即 Dispthread；
- fastboot_status_thread：状态机切换线程；

启动解码内部工作线程：

- ParserThread：解析源码流文件，提取裸流数据，传递给解码器解码线程；

版权所有 © 珠海全志科技股份有限公司。保留一切权利

- DecThread: 解码线程，解码后把输出 buf 传递给送显线程；
- DispThread：送显线程，将解码后的 buffer 调用 disp 驱动接口送显
- preview_layer_config_update（void * frame）；
- preview_display(void);

送显调用以上两个接口，preview_layer_config_update 更新送显的 buf，preview_display 等待 vsync 更新。

最后动画播放结束后，risc-v 会把 ve 的硬件控制权如 iommu，中断，时钟归还给 arm:

2.1.3 异构核固件加载引导流程介绍

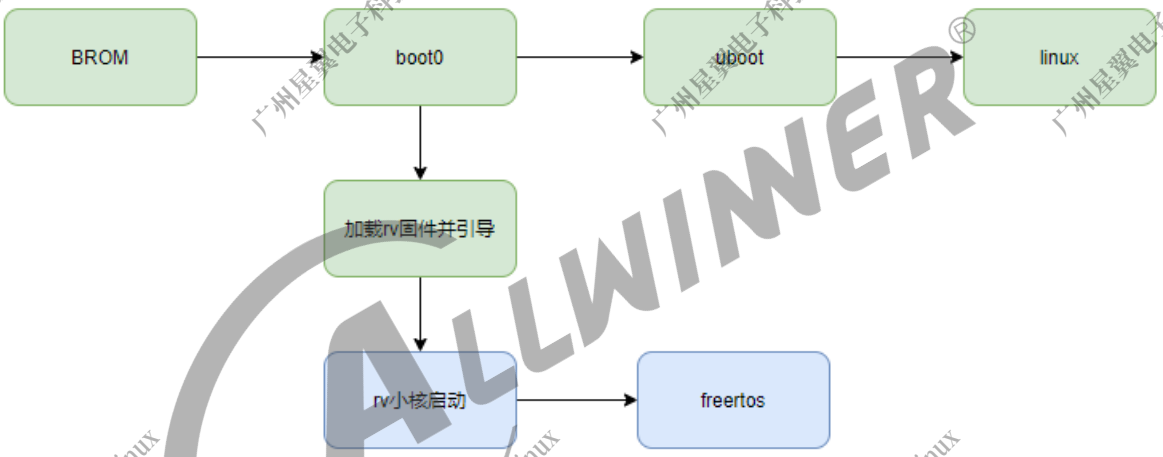

图 2-6: rv 快启动流程

如上图所示，绿色框代表 arm 核的动作，蓝色代表 riscv 核的动作。系统开机后，启动 arm 核，并在 boot0 阶段从 boot_package 中加载 rv 固件，设定 rv 核的起始运行地址，rv 核启动后会从该地址执行，最终执行 freertos 系统，带起 freertos 系统, arm 核则继续正常启动流程, 相关配置和引导流程如下：

```
e906_mem_fw: e906_mem_fw@5F000000 {
    /* boot0 & uboot0 load elf addr */
    reg = <0x0 0x5F000000 0x0 0x00a00000>;
};
```

在 dts 中需要预留 boot0，rv 固件的加载地址，默认为 0x5f000000（这里以 T113 c906 为例，e907 和 a73 核同样需要预留固件加载地址）。

riscv 被打包到 boot_package 中，需要修改 boot_package 的 item 条目的配置文件，位于 configs 目录对应板级目录下

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
[package]
;item=Item_TOC_name,          Item_filename,
item=u-boot,                   u-boot.fex
item=monitor,                  monitor.fex
item=scp,                      scp.fex
item=melis-elf,                amp_rv0.fex
;item=dtbo,                   p1.dtbo
;item=logo,                   bootlogo.bmp.lzma
;item=shutdowncharge,         bempty.bmp.lzma
;item=androidcharge,          battery_charge.bmp.lzma
item=dtb,                     sunxi.fex
```

图 2-7: boot_package.cfg 配置

riscv 固件一般被定义为 amp_rv0.fex, 同构异构核定义为 amp_arm64.bin

在 spl 仓库需要增加一些配置

sdk 目录 brandy/brandy-2.0/spl/board/sun55iw3p1/common.mk 中要添加对应 rv 快速启动 (倒车) 的编译配置

```
BOARD_BUILD_NBOOT = mmcfastboot mmc_car_fastboot nandfastboot
```

- mmcfastboot 是公共平台 boot0 快起的配置
- nandfastboot 是公共平台 nand 版型 boot0 快起的配置
- mmc_car_fastboot 对应是 rv 快速倒车 boot0 快起的相关配置，保存在 mmc_car_fastboot.mk 中，本文档关注的主要是这个配置文件，这几个选项需要打开。

```
CFG_RISCV_E906=y  //加载并引导rv小核的选项
CFG_SUNXI_ELF=y   //引导elf文件的接口
CFG_MELISELF_LOAD_ADDR=0x5F000000 //rv固件的加载地址
CFG_SUNXI_HWSPINLOCK //需要boot0提前初始化好hwspinlock模块
```

最终编译会生成 boot0_mmc_car_fastboot.bin 的 boot0 镜像，打包脚本将其打包到最终镜像中。

boot0 入口函数 boot0_main.c 中，通过 load_image 把 rv 固件加载到 ddr 预留地址中，通过 boot_riscv 接口引导 freertos 启动。

```
  status = load_package();
  if (status == 0)
    load_image(&uboot_base, &optee_base, &monitor_base, &rtos_base, &opensbi_base, &cpus_rtos_base);
  else
    goto _BOOT_ERROR;
#if defined(CFG_RISCV_E907) || defined(CFG_RISCV_E906) || defined(CFG_RISCV_C906)
#ifdef CFG_SUNXI_FDT
  extern void boot_riscv(phys_addr_t base, unsigned long fdt_addr);
  if (cpus_rtos_base != 0)
    boot_riscv(cpus_rtos_base, (unsigned long)working_fdt);
#endif
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
#endif
```

2.1.4 UBOOT 显示图层同步介绍

快速启动异构核显示驱动分两个版本 disp2 和 drm，不同平台适配如下：

ic 平台	显示驱动框架版本
T113	disp2
T527	disp2
A733/t736	drm

快速倒车需要考虑显示硬件控制权在 uboot，rv，内核的轮转，针对 disp2 驱动和 drm 驱动有不同管理机制，以下区分讨论：

2.1.4.1 DISP 驱动版本

uboot 和 riscv 相关的部分主要是显示图层的同步。

以 T527 为例,uboot 配置中 sun55iw3p1_linux_car_defconfig 中要打开以下两个配置：

- CONFIG_DISP2_AMP_PREVIEW_DISPLAY：使能图层信息交换的流程；
- CONFIG_DISP2_PARA_ELF_ADDR=0x60000000：图层信息交换的预留地址；

riscv 和 uboot 图层信息交换通过 hwspinlock 来实现，所以需要在 boot0 先提前初始化好 hwspinlock 驱动，打开配置 CFG_SUNXI_HWSPINLOCK。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

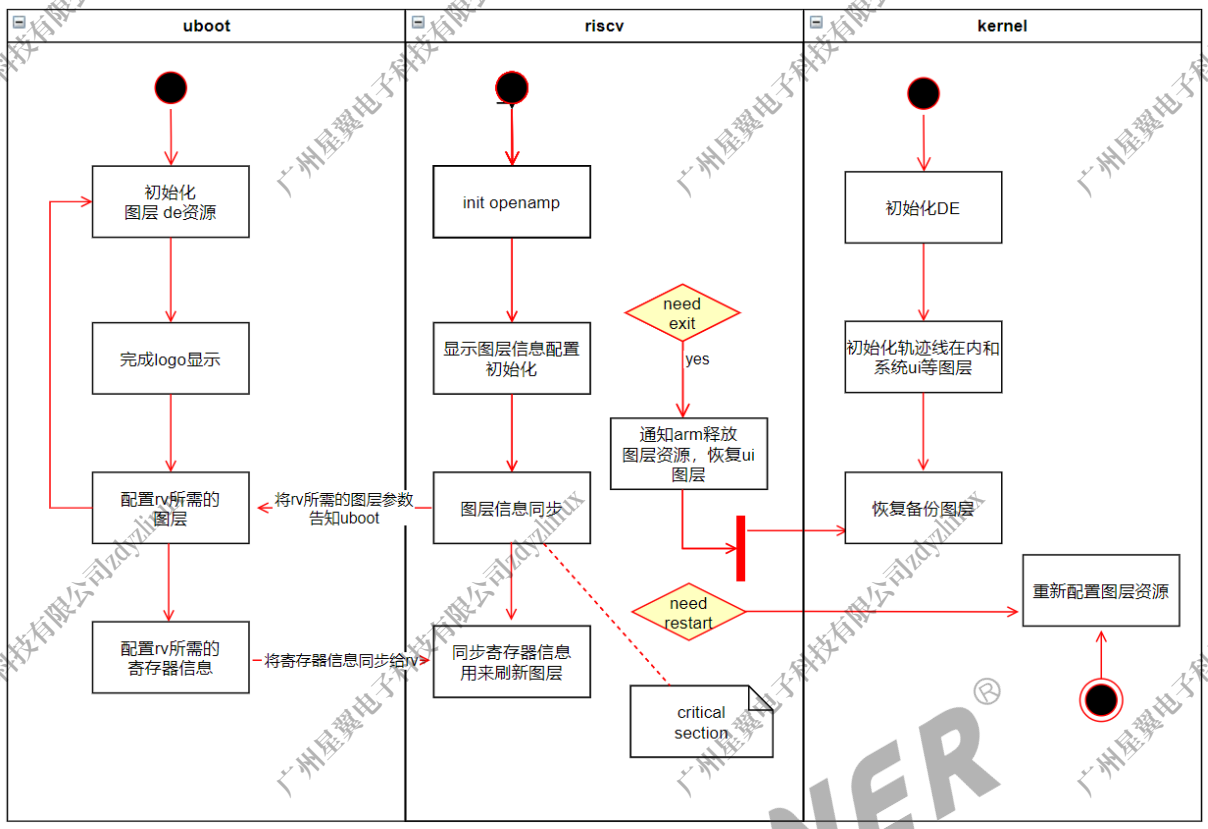

图 2-8: uboot 图层信息同步

disp2 驱动版本 uboot 图层信息同步如上图：

uboot 显示驱动最早执行，完成 logo 显示后通过预留内存及 hw_spinlock 的方式与 riscv 上的 freertos 显示驱动通信，freertos 将所需要显示的图层参数（buffer 宽高格式，显示区域等）通过写预留内存的方式通知 uboot，uboot 拿到图层信息后，将相对应的 channal 配置好，且 zorder 处于 bootlogo 图层之下，即用户不可见，并将 RISCV 使用的 channal 寄存器信息通过写预留内存的方式完成通知，随后不再操作触碰显示硬件寄存器。

freeRTOS 显示驱动拿到 DE 操作图层的寄存器后，根据需要可以使用 uboot 初始化好的所使用的图层 channal 将需要显示的画面显示出来，使用完毕后则恢复原样使得画面继续显示 bootlogo。并在 open_amp 通信建立后（即 linux 显示驱动就绪）告知当前是否在使用显示硬件。

linux 显示驱动操作硬件前会通过 rpmsg 与 freeRTOS 确认所有权，仅当 freeRTOS 没在使用显示硬件时，才响应 linux 端的显示请求。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2.1.4.2 DRM 驱动版本

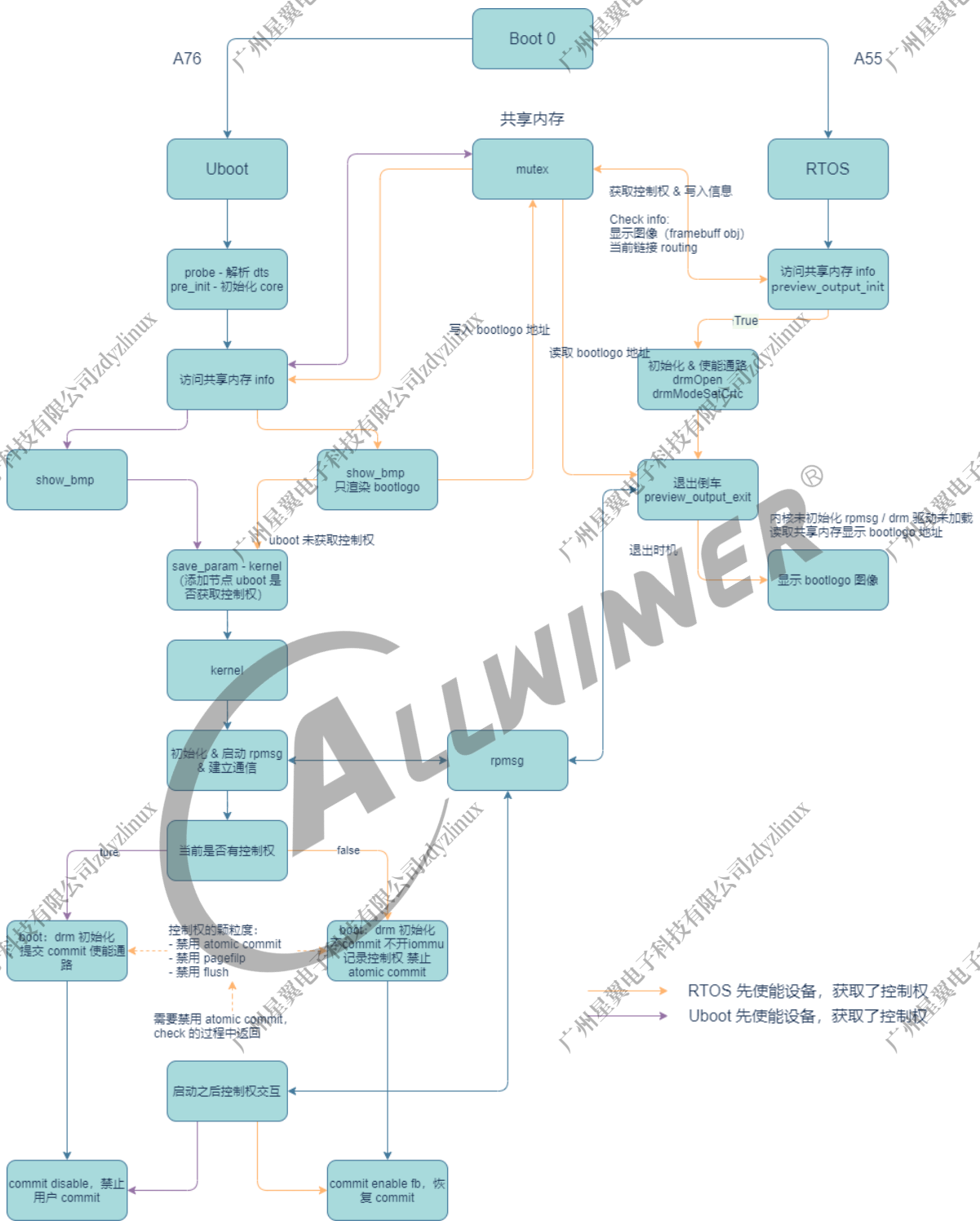

图 2-9: drm 版本显示控制权管理

drm 驱动版本，需要考虑到 uboot，同构核 rtos，以及内核之间的三方交互：

- 启动时 uboot 和 rtos 通过硬件锁共享内存同步当前控制权情况，并写到设备树中，以通知内

版权所有 © 珠海全志科技股份有限公司。保留一切权利

核。

- 内核根据设备树节点判断是否需要对 drm commit 使能通路。
- 内核 rpmsg 组件初始化后，通过 rpmsg 端点同步当前内核 arm 驱动和 rtos drm 驱动的控制权。

2.1.5 RISCV 快启任务及线程介绍

freertos 引导启动后，会启动几个线程来处理业务流程，关系图如下

- 线程 fastboot_disp_init_thread：该线程主要用于与 uboot 进行图层同步，由于图层同步是使用 hwspinlock 来进行同步操作，所以需要创建线程来处理，用以免阻塞主线程业务；
- 线程 fastboot_status_thread：该线程主要用于配置默认快启状态，以及快启状态的监控；对于 rv 倒车默认可以配置为 CAR_REVERSE_START, CAR_REVERSE_STOP, 即默认启动或者关闭倒车，开启 CONFIG_CAR_REVERSE_STABILITY_TEST 可以循环进出倒车进行老化测试，开启 CONFIG_CAR_REVERSE_GPIO_CHECK 配置对于的 gpio 可以通过控制倒车线进出倒车；对于开机动画，默认配置为 CAR_VIDEO_START，默认开启开机动画；
- 函数 fastboo_rpmsg_init：该函数在 openamp 初始化后被调用，主要是用于异步打开 master arm 端的 rpmsg ept 断点建立通信。

1. preview_display_rpmsg_init：建立与 arm disp 显示驱动的 rpmsg 服务，并且同步控制权在 rv 侧；
2. car_reverse_amp_rpmsg_init: 建立与 arm 端倒车驱动的 rpmsg 服务；
3. video_source_rpmsg_init：建立与 arm 端和视频采集相关模块的 rpmsg 服务；
4. preview_enhancer_rpmsg_init: 建立与 arm 端 deinterlace 驱动的 rpmsg 服务；

- 函数 fastboot_init：资源初始化工作，倒车或动画的配置资源在这里申请。

1. preview_display_init：显示图层初始化；
2. fastboot_parse_config：快启参数解析，包括视频输入类型，快启业务类型（倒车/动画），是否旋转，是否开启 deinterlace，屏幕尺寸，输入源分辨率等，这些配置大多可以在 freertos 的 defconfig 中配置；
3. buffer_pool_mem_init：倒车内存池初始化，会从 ddr 从预留内存中申请，对应 dts 中:

```
car_reverse_reserved: car_reverse_reserved@61400000 {
    reg = <0x0 0x61400000 0x0 0x01900000>;
};
```

4. car_reverse_auxiliary_line_buffer_alloc：从刚刚创建的内存池中，配置倒车轨迹线的内存;

版权所有 © 珠海全志科技股份有限公司。保留一切权利

5. car_reverse_video_source_init：视频输入源回调注册，把 vin 或 tvd 的回调注册到 rv 倒车驱动的回调中。
6. ve_buffer_pool_mem_init：申请 ve 解码的预留内存堆；
7. rv_dec_init：初始化解码器并创建解码业务线程；
8. start_decode: 开启解码；

- fastboot_thread: 核心业务线程，监听消息队列，根据消息类型处理业务，**新增的功能可以在此线程中添加**，有以下几种消息类型：

1. 各模块 rpmsg 服务创建成功消息，包括 vin，disp，car_reverse, di 等；
2. 切换状态消息，线程 fastboot_status_thread 切换状态后会通过消息队列通知 fastboot_thread 线程解析后处理对应的流程；
3. arm 消息类型：arm 会发送控制消息给 rv（可以通过），消息的枚举保存在 arm_car_reverse_status；

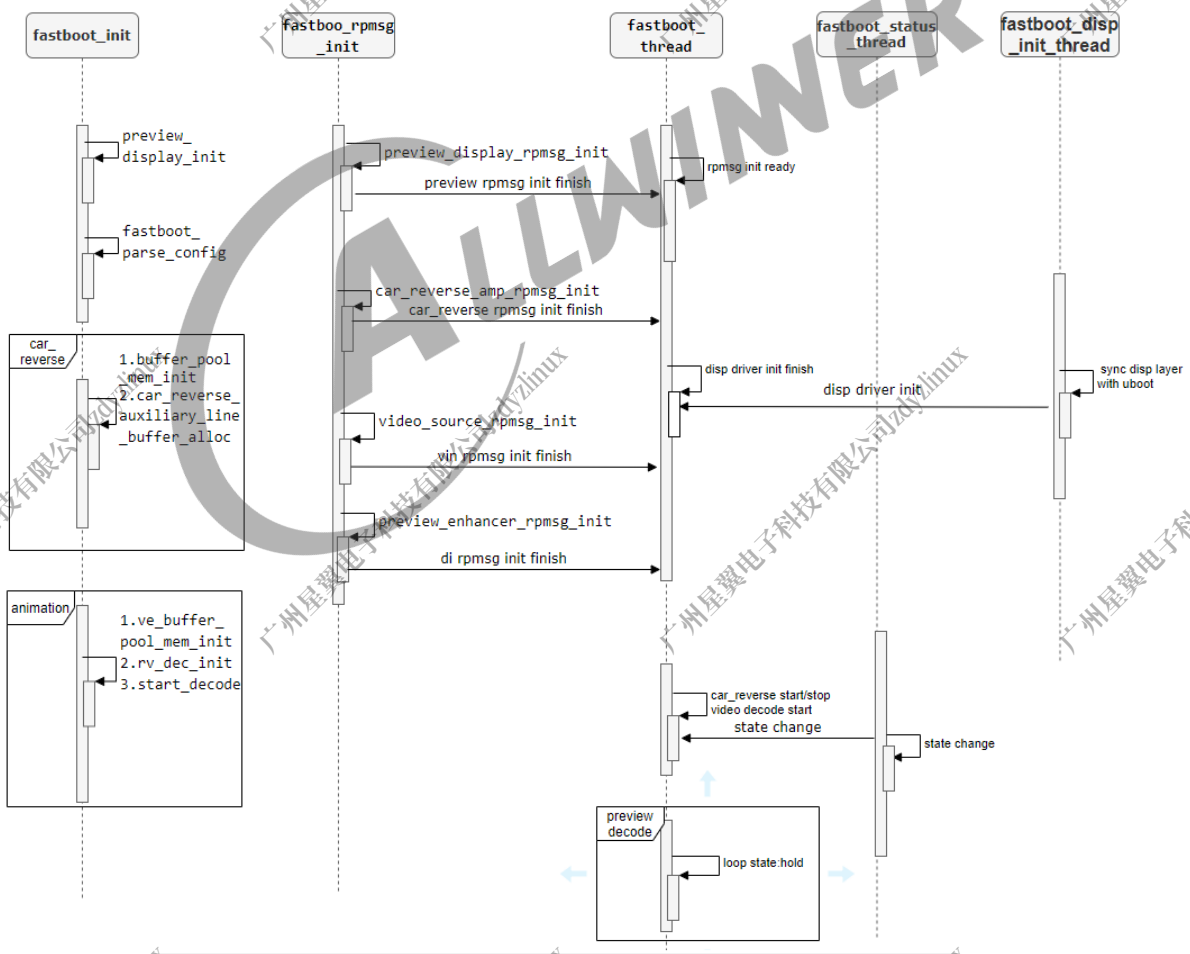

图 2-10: 快启任务间消息处理

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2.1.6 buf 数据流介绍

2.1.6.1 快速倒车

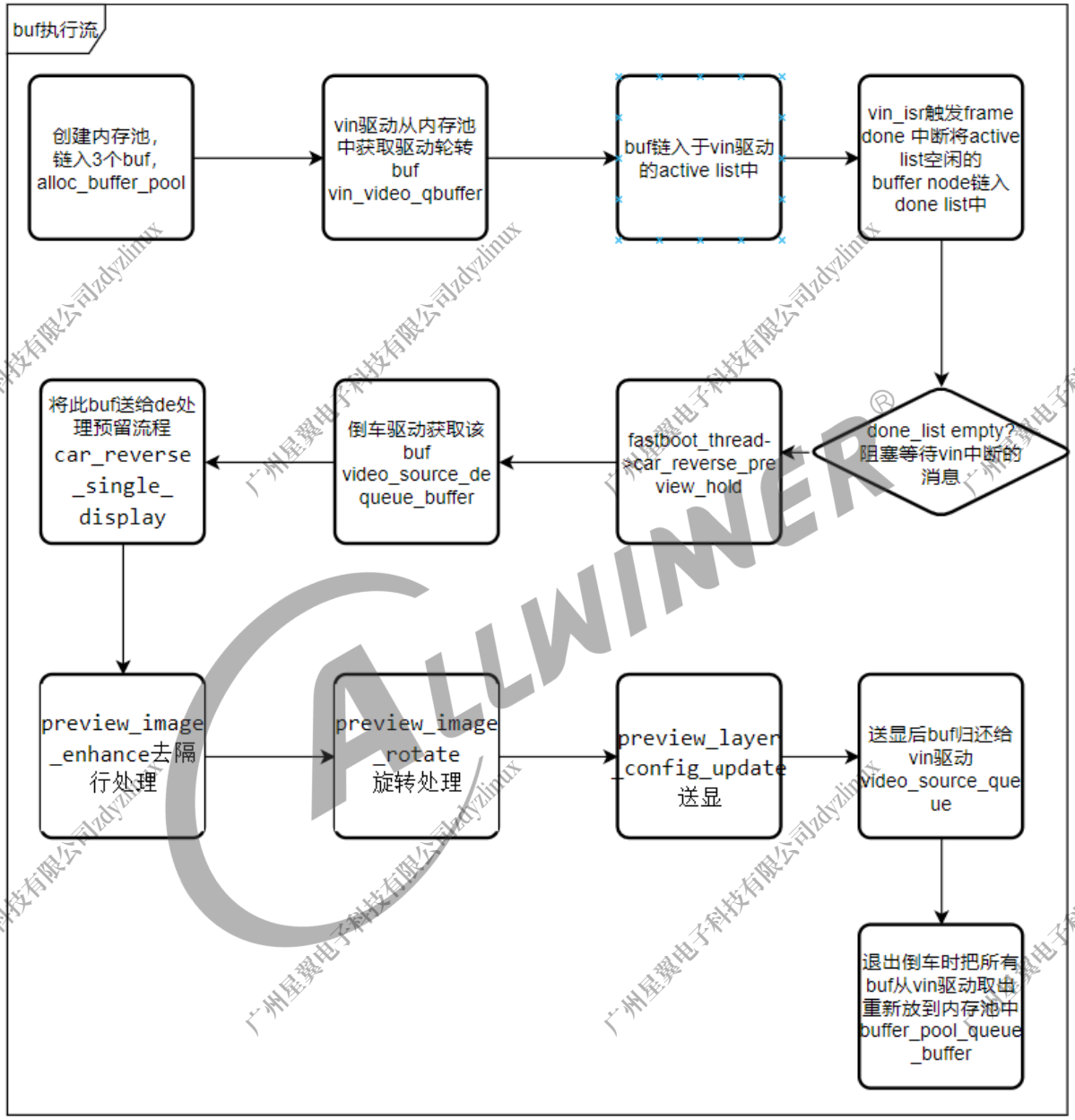

图 2-11: 倒车 buf 执行流

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2.1.6.2 快速动画

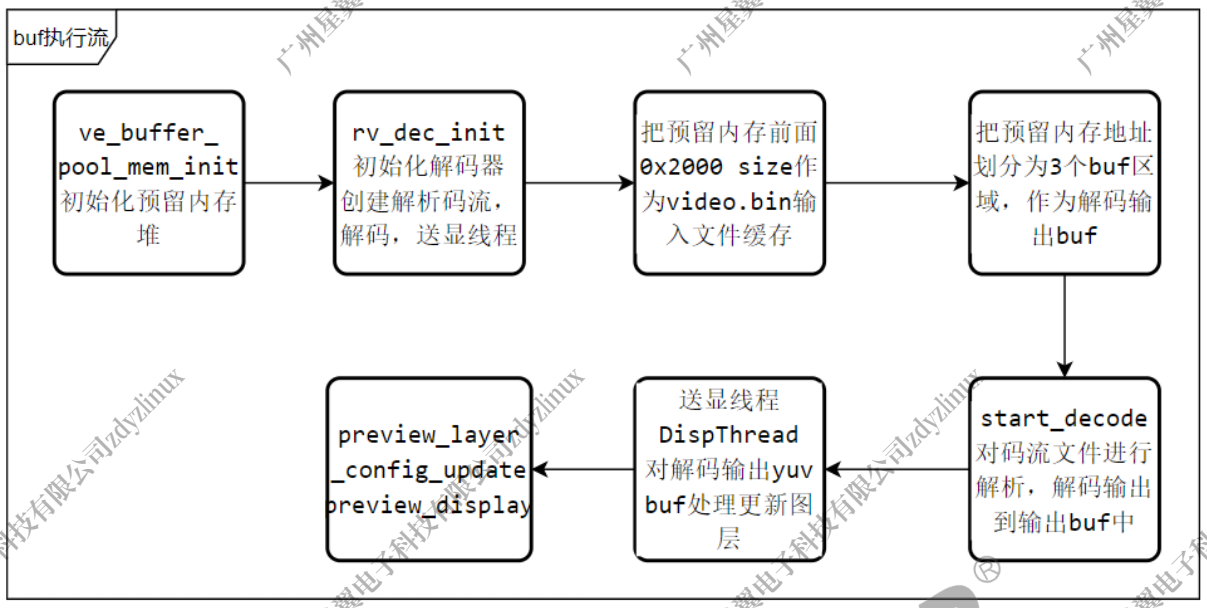

图 2-12: 快速动画 buf 执行流

2.1.7 倒车轨迹线

倒车轨迹线是指在倒车过程中，根据方向盘返回参数，用单独的 rgb 图层绘制出辅助驾驶的轨迹线在倒车预览画面上，以下是不通平台的适配情况。

ic 平台	轨迹线绘制位置	是否支持动态调节角度
T527/T113	uboot	否
A733/t736	a73 核	是

动态轨迹线调整接口

int set_aux_line_lr(int argc, char *argv[])

- 函数原型：int set_aux_line_lr((int argc, char *argv[])
- 功能描述：配置轨迹线方向；
- 参数说明：命令行传参个数 1，0 为配置为左方向，1 为配置为右方向；
- 返回值：成功：返回 0 失败：-1

int set_aux_line_rotate(int argc, char *argv[])

版权所有 © 珠海全志科技股份有限公司。保留一切权利

- 函数原型：int set_aux_line_lr((int depth, char *argv[])
- 功能描述：配置轨迹线角度；
- 参数说明：命令行传参个数 1，轨迹线角度值 0~10；
- 返回值：成功：返回 0 失败：-1

2.2 源码结构介绍

```
//longan/rtos/lichee/rtos/drivers/rtos-hal/hal/source/car_reverse
├── buffer_pool.c
├── buffer_pool.h
├── car_reverse_amp.c
├── car_reverse_amp.h
├── car_reverse.c
├── car_reverse.h
├── drm_preview.c
├── car_reverse_test.c
├── car_reverse_test.h
├── fastboot_com.c
├── fastboot_com.h
├── fastboot_video.c
├── fastboot_video.h
├── include
│   └── libcedarc
│       ├── cdc_config.h
│       ├── cdc_log.h
│       ├── CdcMessageQueue.h
│       ├── cdc_syscall.h
│       ├── CdcSysinfo.h
│       ├── CdcUtil.h
│       ├── cdc_version.h
│       ├── gralloc_metadata
│       │   └── sunxi_metadata_def.h
│       ├── libcedarc.h
│       ├── memoryAdapter.h
│       ├── sc_interface.h
│       ├── sdecoder.h
│       ├── typedef.h
│       ├── vdecoder.h
│       ├── veAdapter.h
│       ├── veInterface.h
│       └── vencoder.h
├── Kconfig
├── Makefile
├── preview_display.c
├── preview_display.h
├── preview_enhancer.c
├── preview_enhancer.h
├── preview_rotator.c
├── rawStreamParser.c
├── rawStreamParser.h
├── rv_decode.c
├── rv_decode.h
├── video_buffer_pool.c
├── video_buffer_pool.h
└── video_source.c
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
└── video_source.h
```

car_reverse.c：快速倒车驱动代码。

car_reverse_amp.c: 倒车流程 rpmsg 消息节点代码，控制倒车状态。

buffer_pool.c：buffer 管理相关代码。

preview_display.c：快速倒车 disp2 显示相关的代码。

drm_preview.c: 快速倒车 drm 显示驱动代码。

fastboot_com.c : 快速启动主要业务处理代码，初始化解析参数

preview_rotator：显示旋转模块的相关代码。

preview_enhancer：显示增强模块的相关代码。

virtual_source：虚拟输入源模块相关代码。

ibcedarc：VE 解码器相关的头文件

2.3 模块配置介绍

2.3.1 rtos menuconfig 配置

进入 sdk 路径下 rtos 目录

1）source envsetup.sh；

2）lunch_rtos 根据版型选择

- T113 平台：选择 T113_s3p_c906_evb1_auto_fastboot 或者 T113_s4_c906_evb1_auto_fastboot_video,
- T527 平台：选择 T527_e906_demo_fastboot_android
- T527v 平台：选择 T527v_e906_demo_fastboot_android

3）mrtos_menuconfig

或者进入对应方案配置目录，如 T527 方案中 longan/rtos/lichee/rtos/projects/T527_e906/demo_fastboot_android，逐个修改配置。

快速倒车相关的配置如下：

- CONFIG_DRIVERS_FASTBOOT：快启程序总开关

版权所有 © 珠海全志科技股份有限公司。保留一切权利

- CONFIG_CAR_REVERSE_LOG_LEVEL: 倒车模块打印等级；
- CONFIG_PREVIEW_SCREEN_WIDTH: 默认预览图层的叠加的缩放宽度；
- CONFIG_PREVIEW_SCREEN_HEIGHT: 默认预览图层的叠加的缩放高度；
- CONFIG_PREVIEW_SOURCE_WIDTH：默认 csi 输入的分辨率宽度；
- CONFIG_PREVIEW_SOURCE_HEIGHT：默认 csi 输入的分辨率宽度；
- CONFIG_PREVIEW_SOURCE_FORMAT_NV12：默认预览的格式为 NV21；
- CONFIG_CAR_REVERSE_DEFAULT_START: 默认启动快起，也就是默认就启动倒车流程；
- CONFIG_CAR_REVERSE_EXCHANGE_DISP2_PARA : riscv 核心与 a7 主核心中的 uboot 交换显示图层信息，默认打开；
- CONFIG_EXCHANGE_DISP2_PARA_LEN：从 uboot 中传递图层信息的长度；
- CONFIG_DRIVERS_CAR_REVERSE: 倒车驱动的开关；
- CONFIG_CAR_REVERSE_SUPPORT_VIN：使用 csi vin 作为倒车输入源；
- CONFIG_CAR_REVERSE_SUPPORT_AUXILIARY_LINE：使能静态轨迹线；
- CONFIG_CAR_REVERSE_SUPPORT_PREVIEW_ROTATOR：使能 g2d 下旋转功能；
- CONFIG_CAR_REVERSE_SUPPORT_PREVIEW_ENHANCER：使能 di 画面增强模块，即去隔行。
- CONFIG_BUFFER_POOL_SWAP_BUFFER_CNT：分配的 camera 的 buf 格式；
- CONFIG_BUFFER_POOL_MEM_ADDR：内存池基地址，申请 buf 从内存池中申请，需要和 dts

中 car_reverse_reserved 预留地址相同；

- CONFIG_BUFFER_POOL_MEM_LEN=0 预留内存池的大小；
- CONFIG_CAR_REVERSE_MEMORY_LEAK_TEST: 开启内存泄漏的检测机制;
- CONFIG_CAR_REVERSE_STABILITY_TEST: 开启循环老化测试模式。

A733/T736 DRM 驱动相关配置（仅需要在 A733/t736 上打开）：

- CONFIG_DRIVERS_SUNXI_DRM : drm 驱动接口开关；
- CONFIG_AW_DRM_DE：使能 sunxi display engine 驱动；
- CONFIG_AW_DRM_TCON：使能 tcon 驱动；
- CONFIG_AW_DRM_TCON_TOP：使能 tcon top 配置驱动；
- CONFIG_AW_DRM_LVDS=y：使能 lvds 接口；
- CONFIG_AW_DRM_TEST_DMABUF_ADDR=0x56004000 ：测试预留地址
- CONFIG_AW_DRM_AMP_COMMUNICATTION：使能 rpmsg 通信接口，管理控制权
- CONFIG_AW_DRM_AMP_SHARE_MEM_ADDR=0x56810000：与 uboot 通信的共享内存地址

版权所有 © 珠海全志科技股份有限公司。保留一切权利

- CONFIG_AW_DRM_AMP_SHARE_MEM_SIZE=0x1000；与 uboot 通信的共享内存长度
- CONFIG_AW_DRM_AMP_RPMSG: 使能异构通信端点；
- CONFIG_AW_DRM_PANEL：dram 平面接口；
- CONFIG_PANEL_LVDS_GENERAL：lvds 类去哦东；
- CONFIG_AW_DRM_PHY: drm 显示 phy 驱动
- CONFIG_AW_DRM_DSI_COMBOPHY：drm 复合 phy 驱动

快速动画的配置如下：

显示相关和快速倒车类似：

- CONFIG_DRIVERS_VIDEO_EARLY：快速动画总开关；
- CONFIG_VIDEO_EARLY_RV_DECODE_SUPPORT：快速动画解码器驱动；
- CONFIG_BUFFER_POOL_VE_MEM_ADDR=0x42700000：快速动画预留首地址；
- CONFIG_BUFFER_POOL_VE_MEM_LEN=0x01700000：快速动画预留地址长度；
- CONFIG_BUFFER_POOL_VE_BUFFER_CNT=5：快速动画送显轮转 buf 个数。

2.3.2 linux 平台配置说明

board.dts 中主要是预留内存相关：

快速倒车相关配置如下：

csi isp 和 vipp 寄存器预留地址：

```
isp_dram_reserved: isp_dram@43e00000 {
    reg = <0x0 0x43e00000 0x0 0x00019000>;
    no-map;
};
```

倒车驱动 buffer 地址：

```
car_reverse_reserved: car_reverse_reserved@45600000 {
    reg = <0x0 0x45600000 0x0 0x02400000>;
    no-map;
};
```

快速动画相关配置如下：

解码器预留地址：

```
ve_ddr_reserved: ve_ddr_reserved@42900000 {
    reg = <0x0 0x42900000 0x0 0x01500000>;
};
```

快速动画流程预留地址：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
video_data: video_data@44000000 {
    reg = <0x0 0x44000000 0x0 0x00200000>;
};
```

- 0x44000000~0x44020000 存放原始 video.bin 文件，128k 大小，注意如果替换文件大于 128k 这里需要修改扩大。
- 0x44020000~0x44200000 存放解码 output buf，最大 5 个 buf。

通用运行预留内存

```
/* 以下适用于T113/T527 ic板级 */
e906_mem_fw: e906_mem_fw@5F000000 {
    /* boot0 & uboot0 load elf addr */
    reg = <0x0 0x5F000000 0x0 0x00a00000>;
};

e906_dram_reserved: e906_dram@60000000 {
    reg = <0x0 0x60000000 0x0 0x00a00000>;
    no-map;
};

/* 以下适用于A733/t736板级 */
a55_mem_fw: a55_mem_fw@5f000000 {
    /* boot0 & uboot0 load elf addr */
    reg = <0x0 0x50000000 0x0 0x00100000>;
};

a55_reserved: a55_ddr@0x51000000 {
    reg = <0x0 0x51000000 0x0 0x00a00000>;
    no-map;
};
```

- e906_mem_fw：riscv 固件的加载地址；
- e906_dram_reserved: riscv 预留的 ddr 运行内存；
- a55_mem_fw: 单核 aarch64 的固件存放地址；
- a55_reserved: 单核 aarch64 的运行内存；

异构通信相关

```
/* 以下适用于T113/T527平台 */
/*
 * The name should be "vdev%dbuffer".
 * Its size should be not less than
 *    RPMSG_BUF_SIZE * (num of buffers in a vring) * 2
 *   = 512 * (num of buffers in a vring) * 2
 */
rv_vdev0buffer: vdev0buffer@4ae00000 {
    compatible = "shared-dma-pool";
    reg = <0x0 0x4ae00000 0x0 0x40000>;
    no-map;
};

/*
 * The name should be "vdev%dvring%d".
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
 * The size of each should be not less than
 *    PAGE_ALIGN(vring_size(num, align))
 *   = PAGE_ALIGN(16 * num + 6 + 2 * num + (pads for align) + 6 + 8 * num)
 *
 * (Please refer to the vring layout in include/uapi/linux/virtio_ring.h)
 */
rv_vdev0vring0: vdev0vring0@4ae40000 {
   reg = <0x0 0x4ae40000 0x0 0x2000>;
   no-map;
};
rv_vdev0vring1: vdev0vring1@4ae42000 {
   reg = <0x0 0x4ae42000 0x0 0x2000>;
   no-map;
};


/* 以下适用于A733/t736板级 */
/*
 * The name should be "vdev%dbuffer".
 * Its size should be not less than
 *    RPMSG_BUF_SIZE * (num of buffers in a vring) * 2
 *   = 512 * (num of buffers in a vring) * 2
 */
a55_vdev0buffer: vdev0buffer@4ae00000 {
   compatible = "shared-dma-pool";
   reg = <0x0 0x4ae00000 0x0 0x40000>;
   no-map;
};

/*
 * The name should be "vdev%dvring%d".
 * The size of each should be not less than
 *    PAGE_ALIGN(vring_size(num, align))
 *   = PAGE_ALIGN(16 * num + 6 + 2 * num + (pads for align) + 6 + 8 * num)
 *
 * (Please refer to the vring layout in include/uapi/linux/virtio_ring.h)
 */
a55_vdev0vring0: vdev0vring0@4ae40000 {
   reg = <0x0 0x4ae40000 0x0 0x2000>;
   no-map;
};
a55_vdev0vring1: vdev0vring1@4ae42000 {
   reg = <0x0 0x4ae42000 0x0 0x2000>;
   no-map;
};
```

- rv_vdev0buffer/a55_vdev0buffer : virtio vdev 共享内存预留地址
- rv_vdev0vringx/a55_vdev0vring: virtio vring0/1 预留地址；

另外还需要定义 e906 或 aarch64 核的 rproc 节点具体如下：

```
e906_rproc: e906_rproc@7130000 {
   mboxes = <&msgbox 8>, <&msgbox 10>;
   mbox-names = "arm-kick", "arm-standby";
   memory-region = <&riscvsram0_reserved>, <&riscvsram1_reserved>, <&rv_vdev0buffer>, <&rv_vdev0vring0>, <&
       rv_vdev0vring1>, <&e906_dram_reserved>;
   auto-boot;
   fw-region = <&e906_mem_fw>;
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
    memory-mappings =
      /* < DA    len    PA >    */
      /* DSP RAM */
      < 0x20000   0x20000    0x20000 >,
      /* SRAM A2 */
      < 0x40000   0x24000    0x40000 >,
      /* DDR */
      < 0x8000000 0x37f00000  0x8000000 >,
      /* SRAM SPACE 0 */
      < 0x3ffc0000    0x40000    0x07280000 >,
      /* SRAM SPACE 1 */
      < 0x40000000    0x40000    0x072c0000 >,
      /* DRAM SPACE */
      < 0x40040000    0x3ffc0000  0x40040000>;
    standby-ctrl-en = <0x1>;
    standby-record-reg = <0x07090114>;
    status = "okay";
  };

  a55_rproc: a55_rproc@0 {
    /* core3 chn0 */
    memory-region = <&a55_vdev0buffer>, <&a55_vdev0vring0>, <&a55_vdev0vring1>, <&a55_reserved>;
    reserve-mem-region = <&car_reverse_reserved>, <&isp_dram_reserved>;
    boot-amp = <&amp0>;     //amp配置节点引用，正常不需要修改。
    fw-region = <&a55_mem_fw>;
    mboxes = <&irq_mbox 1 0>; //使用软件msgbox
    mbox-names = "arm-kick";
    auto-boot;          //默认boot0引导需要打开
    core-id = <7>;
    memory-mappings =
    /* DDR front 1GB */
    < 0x40000000    0x40000000     0x40000000 >;
    status = "okay";
  };
```

2.3.3 模块驱动配置

由于 riscv 快启，一开始的硬件控制权是由 riscv 接管的，所以内核模块加载时候需要保证不会操作到时钟，中断，复用引脚，iommu 等硬件资源，对应的模块配置，如 T527 linux 平台为例，内核配置如下：

- CONFIG_AW_G2D：riscv 倒车用到 g2d 需要编译入内核，保证不初始化硬件；
- CONFIG_G2D_USE_HWSPINLOCK：使能 g2d arm 和 riscv 之间硬件锁同步机制；
- CONFIG_SUNXI_DI_AMP_SERVER：riscv 倒车使用到 deinterlace 去隔行模块，打开与 riscv 的 amp 通信接口开关；
- CONFIG_VIN_INIT_MELIS：riscv：倒车使用到 vin 模块，打开该配置不出初始化硬件资源；
- CONFIG_RV_RUN_CAR_REVERSE：使能 vin 驱动倒车相关处理接口；
- CONFIG_VIDEO_SUNXI_CAR_REVERSE_AMP：使能 riscv 倒车状态控制驱动；
- CONFIG_AW_TWI_DELAYINIT：riscv 倒车使用到 twi 模块，内核初始化时让出控制权；
- CONFIG_AW_RPROC_FAST_BOOT：boot0 或 uboot 启动需要配置开启这个宏；
- CONFIG_AW_RPMSG_NOTIFY：开启倒车流程 rpmsg 通信节点。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

A733/t736 drm 驱动相关配置

- CONFIG_AW_DRM_RPMSG: 使能 drm 驱动下的 rpmsg driver 用于和 rtos 通信；

dts 相关配置：

- twi 节点：rproc-name = “7130000.e906_rproc” 增加该属性支持和 riscv 的控制权共享，

::: warning twi 增加的节点，取决于异构核 rtos 使能的是哪一路 twi，默认公版使用的都是 twi2/3，所以在 dts 的

twi2/twi3 节点都需要，增加 rproc-name 属性；

另外在 rtos/lichee/rtos-hal/source/vin/vin_rpmsg.c 中

```
void vin_return_control(int type)
{
    char * msg = NULL;
    int len = 0;

    if(!type) {
        msg = "get";
        len = 3;
    } else {
        msg = "return";
        len = 6;
    }

    vin_rpmsg_send("twi2", msg, len); //根据具体使用的twi修改
    vin_rpmsg_send("twi3", msg, len); //根据具体使用的twi修改
    vin_rpmsg_send("isp0", msg, len);
    vin_rpmsg_send("scaler0", msg, len);
    vin_rpmsg_send("scaler4", msg, len);
    vin_rpmsg_send("scaler8", msg, len);
    vin_rpmsg_send("scaler12", msg, len);
    vin_rpmsg_send("vinc0", msg, len);
    vin_rpmsg_send("vinc4", msg, len);
    vin_rpmsg_send("vinc8", msg, len);
    vin_rpmsg_send("vinc12", msg, len);
    vin_rpmsg_send("vinc16", msg, len);
    vin_rpmsg_send("vinc17", msg, len);
}
```

:::

- vind 节点：在对应通路的 isp，scalce，sensor，vinc 节点中增加 delay_init 属性，支持和 riscv 的控制权共享，这部分 sdk 已经默认配置好。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2.4 公共接口 API 介绍

2.4.1 视频输入回调

video_source：抽象 csi vin 和 tvd 的对外接口，提供给快启程序使用，接口流程基本为 v4l2 标准的采集流程；

```
//rtos\lichee\rtos-hal\hal\source\car_reverse\video_source.h
struct car_reverse_video_source {
    char *type;
    int (*video_open)(int video_id);
    int (*video_close)(int video_id);
    int (*video_info)(void);
    int (*video_streamon)(int video_id);
    int (*video_streamoff)(int video_id);
    int (*queue_buffer)(int video_id, struct video_source_buffer *buf);
    int (*dequeue_buffer)(int video_id, struct video_source_buffer **buf);
    int (*force_reset_buffer)(int video_id);
    int (*video_set_format)(int video_id, struct vin_format *fmt);
    int (*video_get_format)(int video_id, struct vin_format *fmt);
    void (*car_reverse_callback)(int video_id);
    int (*video_rpmsg_init)(void);
    int (*video_rpmsg_status_on_off)(enum video_packet_type type);
    int (*video_rpmsg_get_status)(void);
    int (*video_source_rpmsg_sem_timewait)(int ms);
};
```

函数名	释义
video_open	打开视频流
video_close	关闭视频流
video_info	获取视频流信息
video_streamon	启动采集
video_streamoff	停止采集
queue_buffer	把 buf 还给驱动
dequeue_buffer	从驱动中获取一帧 buf
force_reset_buffer	把所有 done list 的 buf 链入 active buf
video_set_format	设置视频流输出格式
video_get_format	获取视频流输出格式
car_reverse_callback	未使用
video_rpmsg_init	初始化 rpmsg 通道

版权所有 © 珠海全志科技股份有限公司。保留一切权利

函数名	释义
video_rpmsg_status_on_off	通知远端处理器目前倒车状态
video_rpmsg_get_status	获取目前 rv 倒车控制权状态
video_source_rpmsg_sem_timewait	等待远端控制器 rpmsg ept 端点创建完成的信号量

以上为 video_source 所封装的回调函数，tvd 驱动或 csi 驱动的外部接口需要注册到这些回调中。

2.4.2 内存相关接口

内存相关接口主要 buffer_pool.c 中实现

buffer_pool_mem_init()

- 函数原型：void buffer_pool_mem_init(void)
- 功能描述：完成预留内存堆的初始化。
- 参数说明：无
- 返回值：无

buffer_mem_deinit()

- 函数原型：void buffer_mem_deinit(void)
- 功能描述：完成预留内存堆的销毁。
- 参数说明：无
- 返回值：无

alloc_buffer_pool()

- 函数原型：struct buffer_pool *alloc_buffer_pool((int depth, int buf_size)
- 功能描述：创建一个内存池。
- 参数说明：depth：内存池存放 buf 的个数；buf_size: buf 大小
- 返回值：成功：返回内存池句柄失败：NULL

free_buffer_pool()

- 函数原型：void free_buffer_pool(struct buffer_pool *bp)
- 功能描述：销毁一个内存池。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

- 参数说明：bp：内存池句柄
- 返回值：无

buffer_pool_dequeue_buffer()

- 函数原型：struct buffer_node buffer_pool_dequeue_buffer(struct buffer_pool bp)
- 功能描述：从内存池中获取一个 buf node。
- 参数说明：bp：内存池句柄
- 返回值：成功：buffer node 节点失败：NULL

buffer_pool_queue_buffer()

- 函数原型：void buffer_pool_queue_buffer(struct buffer_pool bp, struct buffer_node node)
- 功能描述：把 buf node 归还给内存池。
- 参数说明：bp：内存池句柄 node：要归还的的 buffer
- 返回值：无

2.4.3 显示预览相关接口

disp2 驱动 hal 层接口文件路径：rtos/lichee/rtos-hal/source/car_reverse/preview_display.c

drm 驱动 hal 层接口文件路径：rtos/lichee/rtos-hal/source/car_reverse/drm_preview.c

两个文件对外接口一致，定义如以下描述，但 drm 驱动封装的接口是 drmlib 的 api。

preview_display_init()

- 函数原型：int preview_display_init(void)
- 功能描述：快速倒车显示模块初始化入口，需要在 freertos 上系统初始化时执行一次。
- 参数说明：无
- 返回值：成功：0 失败：-1

preview_display_rpmsg_init()

- 函数原型：int preview_display_rpmsg_init(void)
- 功能描述：快速倒车显示模块 rpmsg 通信相关初始化入口，需要在 openamp_init 执行后调用，且必须在 preview_display_init 执行完毕后才能调用。只能执行一次。
- 参数说明：无
- 返回值：成功：0 失败：-1

版权所有 © 珠海全志科技股份有限公司。保留一切权利

car_reverse_preview_display_init()

- 函数原型：int car_reverse_preview_display_init(struct preview_params p, void aux_line);
- 功能描述：快启初始显示模块，可以与 preview_display_rpmsg_init 异步执行，该接口只能被调用一次。
- 参数说明：p: yuv channel 的送显参数 aux_line: 为倒车辅助线图层的 buffer 地址
- 返回值：成功：0 失败：-1

preview_output_init()

- 函数原型：int preview_display_rpmsg_init(void)
- 功能描述：取显示硬件所有权接口，在真正送显前调用，向 arm 端取得显示硬件的操作控制权限。可以被多次执行，但需要与 preview_output_exit 配对使用。
- 参数说明：无
- 返回值：成功：0 失败：-1

preview_layer_config_update

- 函数原型：void preview_layer_config_update(struct buffer_node *frame);
- 功能描述：配置需要显示的新帧的 buffer 地址。
- 参数说明：frame：buffer node 实例
- 返回值：成功：0 失败：-1

preview_display

- 函数原型：void preview_display(void);
- 功能描述：将上一次 preview_layer_config_update 配置的新帧地址送入显示，并在开始显示后函数才返回。
- 参数说明：无
- 返回值：成功：0 失败：-1

preview_output_exit

- 函数原型：void preview_output_exit(void);
- 功能描述：归还显示硬件所有权，RISCV 端退出显示。
- 参数说明：无
- 返回值：成功：0 失败：-1

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3 FAQ

3.1 倒车状态节点

倒车状态可以通过内核节点/sys/class/car_reverse/car_reverse/debug/force_start 节点控制：

- 开始倒车：echo start > force_start；
- 结束倒车：echo stop > force_start;

3.2 新增快启模块

见《RISCV 快启任务及线程介绍》章节，若新增快启模块（usb，pcie）可以在以下线程任务新增接口：* fastboot_init: 在此函数进行模块资源初始化，配置参数解析；* fastboot_rpmsg_init: 在此函数打开 rpmsg ept 端点，建立模块与 arm 的 rpmsg 通信；* fastboot_status_thread: 在此线程监测模块的状态变化；* fastboot_thread: 在此线程进行核心的业务处理，接收相应其他任务的消息，并做处理；

3.3 新增快启版型

由于目前平台版型配置较多，客户所适配的版型并不一定带 riscv 快启的配置，需要进行移植配置，以 T113 s4 芯片新增 nand 版型为例，步骤为

1. 进入对应板级配置目录，T113-tina/device/config/chips/T113_s4/configs 拷贝一份新的 evb1_auto_nand 目录为 evb1_auto_nand_fastboot_video
2. 拷贝该目录主要确认一下几点配置，BoardConfig.mk 中：

- LICHEE_BRANDY_DEFCONF：需要使用对应的快启的 uoot 配置文件，以 T113 快速动画为例，使用的配置名为 sun8iw20p1_auto_fastboot_defconfig；此 uboot 配置会打开 CONFIG_DISP2_AMP_PREVIEW_DISPLAY 和 CONFIG_DISP2_PARA_ELF_ADDR 配置，即会在 uboot 阶段和 riscv 做图层信息的交换。
- LICHEE_RTOS_PROJECT_NAME：用于选择 freertos 编译的版型方案，快速动画选择 T113_s4_c906_evb1_auto_fastboot_video；

版权所有 © 珠海全志科技股份有限公司。保留一切权利

- LICHEE_BOOT0_BIN_NAME：用于选择打包的 boot0 的镜像文件，需要选择打开 boot0 引导 riscv 的镜像，T113 中 nand 的快启版型为 nand_car_fastboot; 具体介绍可以查看模块功能介绍中《RISCV 固件加载引导流程介绍》章节。

3. 在 boot_package.cfg 确认新增条目 item=melis-elf, amp_rv0.fex
4. board.dts 以及内核配置可以查看《模块配置介绍》SOC 平台配置，这些配置复制版型后保持不变即可。
5. 在复制的板级配置目录中增加客户自己的定制化配置，比如不同的储存介质等；
6. 重新编译内核，在 longan 目录中重新./build.sh config 选择新增的版型方案

3.4 替换快启 camera

1. 在 linux 调试新的 camera 对片 csi 出图；
2. 将驱动移植到 lichee/rtos-hal/hal/source/vin/modules/sensor/，主要适配 sensor_fuc_core 结构体中的回调, 以 tp2815 为例：

```
struct sensor_fuc_core tp2815_core  = {
    .g_mbus_config = sensor_g_mbus_config,
    .sensor_test_i2c = sensor_test_i2c,
    .sensor_power = sensor_power,
    .s_ir_status = sensor_set_ir,
    .s_stream = sensor_s_stream,
    .s_switch = sensor_s_switch,
    .s_exp_gain = sensor_s_exp_gain,
    .sensor_g_format = sensor_get_format,
    .sensor_g_switch_format = sensor_get_switch_format,
};
```

3. 在该目录下 camera.h 增加 extern 该结构体 sensor_func_core

```
extern struct sensor_fuc_core tp2815_core;
```

4. 注册 sensor 驱动的的 func_core 结构体

lichee/rtos-hal/hal/source/vin/modules/sensor/sensor_register.c

```
#ifdef CONFIG_SENSOR_TP2815_MIPI
    {"tp2815_mipi", &tp2815_core},
#endif
```

5. 修改 vin 配置文件，以 T527 tp2815 为例，位于 T527/longan/rtos/lichee/rtos/drivers/rtos-hal/hal/source/vin/platform/vin_config_sun55iw3.c

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
struct sensor_list global_sensors[VIN_MAX_CSI] = {
if defined CONFIG_SENSOR_TP2815_MIPI
    .used = 1,
    .sensor_name = "tp2815_mipi",
    .sensor_twi_addr = 0x88,
    .sensor_twi_id = 2,
    .mclk_id = 0,
    .use_isp = 0,
    .id = 0,
    .addr_width = 8,
    .data_width = 8,
    .reset_gpio = 0xffff,
    .pwdn_gpio = 0xffff,
    .ir_cut_gpio[0] = 0xffff, /*-cut*/
    .ir_cut_gpio[1] = 0xffff,  /*+cut*/
    .ir_led_gpio = 0xffff, //GPIOE(10)
#endif
struct vin_core global_video[VIN_MAX_VIDEO] = {
if defined CONFIG_SENSOR_TP2815_MIPI
    .used = 1,
    .id = 0,
    .rear_sensor = 0,
    .front_sensor = 0,
    .csi_sel = 0,
    .mipi_sel = 0,
    .isp_sel = 4,
    .tdm_rx_sel = 0xff,
    .isp_tx_ch = 0,
    .base = CSI_DMA0_REG_BASE,
    .irq = SUNXI_IRQ_CSIC_DMA0,
    .o_width = 1920,
    .o_height = 1080,
    .fourcc = V4L2_PIX_FMT_NV12,
    .use_sensor_list = 0,
    .sensor_lane = 4,
#endif
```

global_sensor 结构配置 mclk，复位，pwdn，twi 寄存器位数，global_video 配置数据链路。

6. 修改 twi 配置 在/longan/rtos/board/T527_e906/demo_fastboot_android/configs/sys_config.fex 修改 twi gpio

3.5 快启时间统计

目前快启时间通过打印时间戳统计，如快速倒车时间及快速动画时间就是 boot0 引导 rv 时间戳 +rv 送显时间时间戳，两处打印位置如下：

brandy/brandy-2.0/spl/nboot/main/boot0_main.c

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
#if defined(CFG_RISCV_E907) || defined(CFG_RISCV_E906) || defined(CFG_RISCV_C906)
#ifdef CFG_SUNXI_FDT
    extern void boot_riscv(phys_addr_t base, unsigned long fdt_addr);
    if (cpus_rtos_base != 0)
        boot_riscv(cpus_rtos_base, (unsigned long)working_fdt);
    sunxi_set_printf_debug_mode(7, 0);
    printf("risc-v run ..\n");
    sunxi_set_printf_debug_mode(BT0_head.prvt_head.debug_mode, 0);
#endif
#endif
```

图 3-1: boot0 引导 freertos 时间戳

“[379]risc-v run..” 即 boot0 引导 rv 的时间戳。

rtos/lichee/rtos-hal/hal/source/car_reverse/car_reverse.c

```
            else
                car->discard_frame = 0;

            if (car->discard_frame == 0) {
                car_reverse_single_display(car, &update_frame, &enhanced_frame);
            }
            if (car->disp_first_flag == 0) {
                CAR_REVERSE_INFO("disp first frame ok, time %ld ms\n", (read_timer()/24000));
                car->disp_first_flag = 1;
            }
        }
        video_source_queue_buffer(node, config->video_channel);
    }
    //mutex_unlock(&car->preview_lock);
```

图 3-2: rv 送显时间戳

“[CAR_INFO]-[car_reverse_preview_hold]-<683>:disp first frame ok, time 656 ms”

379+656 =1035ms 即启动获取到第一帧送显的时间。

3.6 TWI 通信失败

twi 属于 rtos 和 linux 的共享资源，快启版型默认 twi 控制权在 rtos 这边，若需要在内核先调试 camera，可以先临时关闭快启修改方法为

T527/T113 为例：

路径文件 longan/device/config/chips/T527/configs/demo_car/android/BoardConfig.mk

```
-LICHEE_BRANDY_DEFCONF:=sun55iw3p1_T527_car_defconfig
+LICHEE_BRANDY_DEFCONF:=sun55iw3p1_T527_defconfig
 LICHEE_RTOS_PROJECT_NAME:=T527_e906_demo_fastboot_android
 LICHEE_BOOT0_BIN_NAME:=mmc_car_fastboot
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
\ No newline at end of file
+#LICHEE_BOOT0_BIN_NAME:=mmc_car_fastboot
```

A733/t736 为例：关闭设备树 a55_rproc 节点 auto-boot 属性，disabled sunxi_amp 节点

```
a55_rproc: a55_rproc@0 {
    /* core3 chn0 */
    memory-region = <&a55_vdev0buffer>, <&a55_vdev0vring0>, <&a55_vdev0vring1>, <&a55_reserved>;
    reserve-mem-region = <&car_reverse_reserved>, <&isp_dram_reserved>;
    boot-amp = <&amp0>;
    fw-region = <&a55_mem_fw>;
    mboxes = <&irq_mbox 1 0>;
    mbox-names = "arm-kick";
    //auto-boot; //关闭此属性
    core-id = <7>;
    memory-mappings =
    /* DDR front 1GB */
    < 0x40000000    0x40000000      0x40000000 >;
    status = "okay";
  };

  sunxi_amp: sunxi-amp {
    compatible = "sunxi,amp";
    status = "disabled"; //修改为disabled
    amp-count = <1>;

    amp0: amp0 {
      cpu-id = <0x700>;
      entry-addr = <0x51000000>;
      load-addr = <0x50000000>;
      arch = <1>; /* 0:arm 1:arm64 */
      image = "amp-arm";
      boot-stage = "boot0";
    };

  };
```

另外需要关闭以下这些驱动模块配置

```
CONFIG_G2D_USE_HWSPINLOCK
CONFIG_VIDEO_SUNXI_VIN_SPECIAL
CONFIG_VIN_INIT_MELIS=y
CONFIG_RV_RUN_CAR_REVERSE=y
CONFIG_AW_TWI_DELAYINIT=y
CONFIG_AW_RPROC_FAST_BOOT=y
```

修改这些配置后可以在内核中先调试 sensor 出图，在进行快启的调试。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

著作权声明

版权所有 ©2025 珠海全志科技股份有限公司。保留一切权利。

本文档及内容受著作权法保护，其著作权由珠海全志科技股份有限公司（“全志”）拥有并保留一切权利。

本文档是全志的原创作品和版权财产，未经全志书面许可，任何单位和个人不得擅自摘抄、复制、修改、发表或传播本文档内容的部分或全部，且不得以任何形式传播。

商标声明

ALLWINNER、ALLWINNER 全志科技、全志科技、ALLWINNER（不完全列举）均为珠海全志科技股份有限公司的商标或者注册商标。在本文档描述的产品中出现的其它商标，产品名称，和服务名称，均由其各自所有人拥有。

免责声明

您购买的产品、服务或特性应受您与珠海全志科技股份有限公司（“全志”）之间签署的商业合同和条款的约束。本文档中描述的全部或部分产品、服务或特性可能不在您所购买或使用的范围内。使用前请认真阅读合同条款和相关说明，并严格遵循本文档的使用说明。您将自行承担任何不当使用行为（包括但不限于如超压，超频，超温使用）造成的不利后果，全志概不负责。

本文档作为使用指导仅供参考。由于产品版本升级或其他原因，本文档内容有可能修改，如有变更，恕不另行通知。全志尽全力在本文档中提供准确的信息，但并不确保内容完全没有错误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或发生侵犯第三方权利事件，全志概不负责。本文档中的所有陈述、信息和建议并不构成任何明示或暗示的保证或承诺。

本文档未以明示或暗示或其他方式授予全志的任何专利或知识产权。在您实施方案或使用产品的过程中，可能需要获得第三方的权利许可。请您自行向第三方权利人获取相关的许可。全志不承担也不代为支付任何关于获取第三方许可的许可费或版税（专利税）。全志不对您所使用的第三方许可技术做出任何保证、赔偿或承担其他义务。